

Sổ tay mẫu mã chota1219

Bản dịch tiếng Việt

1 1 Mẫu khung chương trình

```
1 import static java.lang.Math.*;
2 import static java.util.Arrays.*;
3 import java.io.*;
4 import java.util.*;
5
6 public class Main {
7     static boolean LOCAL = System.getSecurityManager() == null;
8     Scanner sc = new Scanner(System.in);
9
10    void run() {
11    }
12
13    void debug(Object...os) {
14        System.err.println(deepToString(os));
15    }
16
17    public static void main(String[] args) {
18        if (LOCAL) {
19            try {
20                System.setIn(new FileInputStream("in.txt"));
21            } catch (Throwable e) {
22                LOCAL = false;
23            }
24        }
25        new Main().run();
26    }
27 }
```

2 2 Cấu trúc dữ liệu

2.1 2.1 BIT

```
1 class BIT {
2     int[] vs;
3     BIT(int n) {
4         vs = new int[n + 1];
5     }
6     void add(int k, int a) {
7         for (int i = k + 1; i < vs.length; i += i & -i) {
8             vs[i] += a;
9         }
10    }
11    int sum(int s, int t) {
12        if (s > 0) return sum(0, t) - sum(0, s);
13        int res = 0;
14        for (int i = t; i > 0; i -= i & -i) {
15            res += vs[i];
16        }
17        return res;
18    }
19    // [0, i] の和が k より大きくなる最小の i を求める
```

```

20 int get(int k) {
21 int p = Integer.highestOneBit(vs.length - 1);
22 for (int q = p; q > 0; q >>= 1, p |= q) {
23 if (p >= vs.length || k < vs[p]) p ^= q;
24 else k -= vs[p];
25 }
26 return p;
27 }
28 }

```

2.2 2.2 RMQ

```

1 class RMQ {
2 int[] vs;
3 int[][] min;
4 RMQ(int[] vs) {
5 int n = vs.length, m = log2(n) + 1;
6 this.vs = vs;
7 min = new int[m][n];
8 for (int i = 0; i < n; i++) min[0][i] = i;
9 for (int i = 1, k = 1; i < m; i++, k <= 1) {
10 for (int j = 0; j + k < n; j++) {
11 min[i][j] = vs[min[i - 1][j]] <= vs[min[i - 1][j + k]] ?
12 min[i - 1][j] : min[i - 1][j + k];
13 }
14 }
15 }
16 //最小値が複数存在する場合は一番最初のを返す
17 int query(int from, int to) {
18 int k = log2(to - from);
19 return vs[min[k][from]] <= vs[min[k][to - (1 << k)]] ?
20 min[k][from] : min[k][to - (1 << k)];
21 }
22 int log2(int b) {
23 return 31 - Integer.numberOfLeadingZeros(b);
24 }
25 }

```

2.3 2.3 Cập nhật đoạn

```

1 class Intervals {
2 TreeMap<Integer, Integer> map = new TreeMap<Integer, Integer>();
3 Intervals() {
4 map.put(Integer.MIN_VALUE, -1);
5 map.put(Integer.MAX_VALUE, -1);
6 }
7 void paint(int s, int t, int c) {
8 int p = get(t);
9 map.subMap(s, t).clear();
10 map.put(s, c);
11 map.put(t, p);
12 }
13 int get(int k) {
14 return map.floorEntry(k).getValue();
15 }
16 }

```

2.4 2.4 Treap

Cài đặt không gây tắc dụng phụ.

```

1 class T {
2 final int key, val;
3 final double p;

```

```

4 final T left, right;
5 T(int key, int val, double p, T left, T right) {
6 this.key = key;
7 this.val = val;
8 this.p = p;
9 this.left = left;
10 this.right = right;
11 }
12 T change(T left, T right) {
13 return new T(key, val, p, left, right);
14 }
15 T normal() {
16 if (left != null && left.p < p && (right == null || left.p < right.p)) {
17 return left.change(left.left, change(left.right, right));
18 } else if (right != null && right.p < p) {
19 return right.change(change(left, right.left), right.right);
20 }
21 return this;
22 }
23 }
24 T put(T t, int key, int val) {
25 if (t == null) return new T(key, val, random(), null, null);
26 if (key < t.key) return t.change(put(t.left, key, val), t.right).normal();
27 if (key > t.key) return t.change(t.left, put(t.right, key, val)).normal();
28 return new T(key, val, t.p, t.left, t.right);
29 }
30 T remove(T t, int key) {
31 if (t == null) return null;
32 if (key < t.key) return t.change(remove(t.left, key), t.right);
33 if (key > t.key) return t.change(t.left, remove(t.right, key));
34 return merge(t.left, t.right);
35 }
36 T merge(T t1, T t2) {
37 if (t1 == null) return t2;
38 if (t2 == null) return t1;
39 if (t1.p < t2.p) return t1.change(t1.left, merge(t1.right, t2));
40 return t2.change(merge(t1, t2.left), t2.right);
41 }

```

2.5 2.5 Cây đoạn

Cài đặt nhanh bằng mảng và vòng lặp.

parent(i) = i/2

left(i) = 2i

size(i) = N/highest(i)

s(i) = size(i)i - N

```

1 class Seg {
2 int N;
3 Seg(int n) {
4 N = Integer.highestOneBit(n) << 1;
5 }
6 //点を含む区間に対する処理
7 void query(int k) {
8 for (int i = N + k; i > 0; i >>= 1) {
9 //...
10 }
11 }
12 //カバーする区間に対する処理
13 void query(int s, int t) {
14 while (0 < s && s + (s & -s) <= t) {
15 int i = (N + s) / (s & -s);
16 //...
17 s += s & -s;
18 }
19 while (s < t) {

```

```

20 int i = (N + t) / (t & -t) - 1;
21 //...
22 t -= t & -t;
23 }
24 }
25 }

```

3 Đồ thị

3.1 Tập công thức

3.1.1 Định lý đa diện Euler

Ghi chú/công thức: $V - E + F = 2$

3.1.2 Định lý max-flow min-cut

Giá trị luồng s-t lớn nhất bằng dung lượng lát cắt s-t nhỏ nhất.

Nếu X là tập các đỉnh đi được từ s trong đồ thị dư G_f , tập cạnh của min-cut trong G là

$\hookrightarrow \delta^+_G(X)$.

$G(X)$

3.1.3 Các hệ thức về matching

$|\text{matching lớn nhất}| \leq |\text{vertex cover nhỏ nhất}|$; với đồ thị hai phía thì có dấu bằng.

$|\text{tập độc lập lớn nhất}| + |\text{vertex cover nhỏ nhất}| = V$.

Với đồ thị không có đỉnh cô lập: $|\text{matching lớn nhất}| + |\text{edge cover nhỏ nhất}| = V$.

Với đồ thị hai phía: $|\text{edge cover nhỏ nhất}| = |\text{tập độc lập lớn nhất}|$.

$X \subset V(G)$ là vertex cover của G khi và chỉ khi $V(G) \setminus X$ là tập độc lập của G .

3.1.4 Định lý cây ma trận

Ghi chú/công thức: $G = (G - G)$

3.1.5 Số cây theo dãy bậc

Trong K_n , số cây khung sao cho đỉnh i có bậc d_i là:

$(n - 2)!$

$(d_1 - 1)!(d_2 - 1)! \dots (d_n - 1)!$

3.1.6 Công thức Kasteleyn

Với đồ thị có thể định hướng mọi chu trình độ dài chẵn theo kiểu lẻ, định thức ma trận kề của nó

$\hookrightarrow$ (cạnh ngược hướng lấy -1)

bằng bình phương số matching hoàn hảo.

Đồ thị phẳng luôn có một định hướng như vậy.

Với đồ thị hai phía, định thức ma trận kề hai phía bằng số matching hoàn hảo.

3.1.7 Ma trận Tutte

Ma trận Tutte là ma trận $V \times V$ thu được bằng cách gán hướng tùy ý cho các cạnh của G để được đồ thị

$\hookrightarrow$ có hướng G' .

Với cạnh (v, u) ,

Ghi chú/công thức: $xvu \text{ } tvu = xvu$, $tuv = -xvu$

Với đồ thị tổng quát, hạng của ma trận Tutte bằng hai lần kích thước matching.

Lấy x ngẫu nhiên cho nghiệm đúng với xác suất đủ cao.

3.2 3.2 Sắp xếp tô pô

Thứ tự tô pô là thứ tự thỏa: nếu có cạnh từ $v[i]$ đến $v[j]$ thì $i < j$.

Nếu đánh số lúc thoát khỏi DFS, ta nhận được thứ tự tô pô đảo.

```
1 V[] topologicalSort(V[] vs) {
2 n = vs.length;
3 us = new V[n];
4 for (V v : vs) {
5 if (v.state == 0 && !dfs(v)) return null;
6 }
7 return us;
8 }
9 boolean dfs(V v) {
10 v.state = 1;
11 for (V u : v) {
12 if (u.state == 1 || u.state == 0 && !dfs(u)) return false;
13 }
14 us[--n] = v;
15 v.state = 2;
16 return true;
17 }
```

3.3 3.3 Phân rã thành phần liên thông mạnh

Trước hết tìm thứ tự tô pô (bỏ qua chu trình), rồi DFS theo cạnh đảo theo thứ tự đó.

Các cây tìm kiếm cực đại thu được khi đó là các thành phần liên thông mạnh.

Trả về số SCC là k ; comp chứa thứ tự tô pô trong $[0, k)$.

```
1 int scc(V[] vs) {
2 n = vs.length;
3 us = new V[n];
4 for (V v : vs) if (!v.visit) dfs(v);
5 for (V v : vs) v.visit = false;
6 for (V u : us) if (!u.visit) dfsrev(u, n++);
7 return n;
8 }
9 void dfs(V v) {
10 v.visit = true;
11 for (V u : v.fs) if (!u.visit) dfs(u);
12 us[--n] = v;
13 }
14 void dfsrev(V v, int k) {
15 v.visit = true;
16 for (V u : v.rs) if (!u.visit) dfsrev(u, k);
17 v.comp = k;
18 }
```

3.4 3.4 Đỉnh khớp và cầu

Chọn gốc tùy ý, chạy DFS và gán số num khi đi vào đỉnh.

Tính low là giá trị nhỏ nhất trong các mục sau:

- num[v].
- num[u] của đỉnh u đã thăm, kề với v, không phải cạnh vừa đi tới.
- low[u] của con u của v.

Khi đó:

Gốc: r là đỉnh khớp khi và chỉ khi r có ít nhất hai con.

Không phải gốc: v là đỉnh khớp khi và chỉ khi tồn tại con u sao cho $\text{num}[v] \leq \text{low}[u]$.

(v, u) là cầu khi và chỉ khi $\text{num}[v] < \text{low}[u]$.

count chứa số phần tách ra khi bỏ đỉnh đó; hàm trả về tập cầu.

```
1 ArrayList<E> connection(V[] vs) {
2     bridge = new ArrayList<E>();
3     for (V v : vs) if (v.num < 0) {
4         dfs(v, 0);
5         if (v.count > 0) v.count--;
6     }
7     return bridge;
8 }
9 int dfs(V v, int c) {
10     v.num = c;
11     int low = c;
12     boolean rev = false;
13     for (V u : v) {
14         if (u.num < 0) {
15             int t = dfs(u, c + 1);
16             low = min(low, t);
17             if (v.num <= t) v.count++;
18             if (v.num < t) bridge.add(new E(v, u));
19         } else if (u.num != v.num - 1 || rev) low = min(low, u.num);
20         else rev = true;
21     }
22     return low;
23 }
```

3.5 3.5 Chu trình có độ dài trung bình nhỏ nhất

Cho phép cạnh âm.

Bất kỳ chu trình nào nằm trên đường đi truy vết bằng prev đều là chu trình có trung bình nhỏ nhất.

$O(V E)$

```
1 double mmc(int[] ss, int[] ts, double[] cs, int n) {
2     int m = ss.length;
3     double[][] dp = new double[n + 1][n];
4     for (int i = 0; i < n; i++) {
5         fill(dp[i + 1], INF);
6         for (int j = 0; j < m; j++) {
7             dp[i + 1][ts[j]] = min(dp[i + 1][ts[j]], dp[i][ss[j]] + cs[j]);
8         }
9     }
10    double res = INF;
11    for (int i = 0; i < n; i++) if (dp[n][i] < INF) {
12        double max = -INF;
13        for (int j = 0; j < n; j++) {
14            max = max(max, (dp[n][i] - dp[j][i]) / (n - j));
15        }
16        res = min(res, max);
17    }
18    return res;
19 }
```

3.6 3.6 Cây có hướng nhỏ nhất với gốc cho trước

Tham lam chọn cạnh vào nhỏ nhất cho mỗi đỉnh khác gốc; nếu sinh chu trình thì co lại và xử lý đệ quy.

Muốn khôi phục cây có hướng, hãy lưu cạnh gốc khi co.

Nếu chưa chỉ định gốc, trừ một giá trị đủ lớn khỏi mọi cạnh, rồi thêm cạnh chi phí 0 từ gốc đến mọi đỉnh khác

là đủ.

Muốn tìm rừng có hướng lớn nhất, đảo dấu trọng số cạnh rồi thêm cạnh chi phí 0 từ gốc đến mọi đỉnh
↪ khác.

$O(V E)$

```
1 int arborescence(V[] vs, V r) {
2 int res = 0;
3 for (V v : vs) for (E e : v.es) e.to.min = min(e.to.min, e.cost);
4 for (V v : vs) if (v != r) {
5 if (v.min == INF) return INF;
6 res += v.min;
7 }
8 for (V v : vs) for (E e : v.es) if (e.to != r) {
9 e.cost -= e.to.min;
10 if (e.cost == 0) {
11 v.fs.add(e.to);
12 e.to.rs.add(v);
13 }
14 }
15 int m = scc(vs);
16 if (m == vs.length) return res;
17 V[] us = new V[m];
18 for (int i = 0; i < m; i++) us[i] = new V();
19 for (V v : vs) for (E e : v.es) {
20 if (v.comp != e.to.comp) us[v.comp].add(us[e.to.comp], e.cost);
21 }
22 return min(INF, res + arborescence(us, us[r.comp]));
23 }
```

3.7 3.7 Cây Steiner

Với đồ thị vô hướng không âm, tìm chi phí cây nhỏ nhất chứa mọi terminal trong ts .

$dp[S][v] :=$ chi phí cây Steiner nhỏ nhất chứa $S \subset T$ và $v \in V$.

$O(3^t$

$n + 2^t$

n^2

$+ n^3$

)

```
1 int steiner(int[] g, int[] ts) {
2 int n = g.length, m = ts.length;
3 if (m < 2) return 0;
4 int[] dp = new int[1 << m][n];
5 for (int k = 0; k < n; k++) {
6 for (int i = 0; i < n; i++) {
7 for (int j = 0; j < n; j++) {
8 g[i][j] = min(g[i][j], g[i][k] + g[k][j]);
9 }
10 }
11 }
12 for (int i = 0; i < m; i++) {
13 for (int j = 0; j < n; j++) {
14 dp[1 << i][j] = g[ts[i]][j];
15 }
16 }
17 for (int i = 1; i < 1 << m; i++) if (((i - 1) & i) != 0) {
18 for (int j = 0; j < n; j++) {
19 dp[i][j] = INF;
20 for (int k = (i - 1) & i; k > 0; k = (k - 1) & i) {
21 dp[i][j] = min(dp[i][j], dp[k][j] + dp[i ^ k][j]);
22 }
23 }
24 for (int j = 0; j < n; j++) {
25 for (int k = 0; k < n; k++) {
```

```

26 dp[i][j] = min(dp[i][j], dp[i][k] + g[k][j]);
27 }
28 }
29 }
30 return dp[(1 << m) - 1][ts[0]];
31 }

```

3.8 3.8 Dinic

```

1 int dinic(V s, V t) {
2 int flow = 0;
3 for (int p = 1; ; p++) {
4 Queue<V> que = new LinkedList<V>();
5 s.level = 0;
6 s.p = p;
7 que.offer(s);
8 while (!que.isEmpty()) {
9 V v = que.poll();
10 v.iter = v.es.size() - 1;
11 for (E e : v.es) if (e.to.p < p && e.cap > 0) {
12 e.to.level = v.level + 1;
13 e.to.p = p;
14 que.offer(e.to);
15 }
16 }
17 if (t.p < p) return flow;
18 for (int f; (f = dfs(s, t, INF)) > 0; ) flow += f;
19 }
20 }
21 int dfs(V v, V t, int f) {
22 if (v == t) return f;
23 for (; v.iter >= 0; v.iter--) {
24 E e = v.es.get(v.iter);
25 if (v.level < e.to.level && e.cap > 0) {
26 int d = dfs(e.to, t, min(f, e.cap));
27 if (d > 0) {
28 e.cap -= d;
29 e.rev.cap += d;
30 return d;
31 }
32 }
33 }
34 return 0;
35 }
36 class V {
37 ArrayList<E> es = new ArrayList<E>();
38 int level, p, iter;
39 void add(V to, int cap) {
40 E e = new E(to, cap), rev = new E(this, 0);
41 e.rev = rev; rev.rev = e;
42 es.add(e); to.es.add(rev);
43 }
44 }
45 class E {
46 V to;
47 E rev;
48 int cap;
49 E(V to, int cap) {
50 this.to = to;
51 this.cap = cap;
52 }
53 }

```


3.9 3.9 Global min-cut nhỏ nhất

Trọng số phải không âm.

$O(n^3)$
)

```
1 int minCut(int[] c) {
2     int n = c.length, cut = INF;
3     int[] id = new int[n], b = new int[n];
4     for (int i = 0; i < n; i++) id[i] = i;
5     for (; n > 1; n--) {
6         fill(b, 0);
7         for (int i = 0; i + 1 < n; i++) {
8             int p = i + 1;
9             for (int j = i + 1; j < n; j++) {
10                b[id[j]] += c[id[i]][id[j]];
11                if (b[id[p]] < b[id[j]]) p = j;
12            }
13            swap(id, i + 1, p);
14        }
15        cut = min(cut, b[id[n - 1]]);
16        for (int i = 0; i < n - 2; i++) {
17            c[id[i]][id[n - 2]] += c[id[i]][id[n - 1]];
18            c[id[n - 2]][id[i]] += c[id[n - 1]][id[i]];
19        }
20    }
21    return cut;
22 }
```

3.10 3.10 Luồng chi phí nhỏ nhất

Đừng quên truyền vào toàn bộ các đỉnh.

```
1 int minCostFlow(V[] vs, V s, V t, int flow) {
2     int res = 0;
3     while (flow > 0) {
4         for (V v : vs) v.min = INF;
5         PriorityQueue<E> que = new PriorityQueue<E>();
6         s.min = 0;
7         que.offer(new E(s, 0, 0));
8         while (!que.isEmpty()) {
9             E crt = que.poll();
10            if (crt.cost == crt.to.min) {
11                for (E e : crt.to.es) {
12                    int tmp = crt.cost + e.cost + crt.to.h - e.to.h;
13                    if (e.cap > 0 && e.to.min > tmp) {
14                        e.to.min = tmp;
15                        e.to.prev = e;
16                        que.offer(new E(e.to, 0, e.to.min));
17                    }
18                }
19            }
20        }
21        if (t.min == INF) return -1;
22        int d = flow;
23        for (E e = t.prev; e != null; e = e.rev.to.prev) {
24            d = min(d, e.cap);
25        }
26        for (E e = t.prev; e != null; e = e.rev.to.prev) {
27            res += d * e.cost;
28            e.cap -= d;
29            e.rev.cap += d;
30        }
31        flow -= d;
32        for (V v : vs) v.h += v.min;
```

```

33 }
34 return res;
35 }
36 class V {
37 ArrayList<E> es = new ArrayList<E>();
38 E prev;
39 int min, h;
40 void add(V to, int cap, int cost) {
41 E e = new E(to, cap, cost), rev = new E(this, 0, -cost);
42 e.rev = rev; rev.rev = e;
43 es.add(e); to.es.add(rev);
44 }
45 }
46 class E implements Comparable<E> {
47 V to;
48 E rev;
49 int cap, cost;
50 E(V to, int cap, int cost) {
51 this.to = to;
52 this.cap = cap;
53 this.cost = cost;
54 }
55 public int compareTo(E o) {
56 return cost - o.cost;
57 }
58 }

```

3.11 Matching hai phía

Vertex cover nhỏ nhất gồm các đỉnh không đi tới được ở phía trái và các đỉnh đi tới được ở phía
 $\hookrightarrow$ phải.

$O(V E)$

```

1 int bipartiteMatching(V[] vs) {
2 int match = 0;
3 for (V v : vs) if (v.pair == null) {
4 for (V u : vs) u.used = false;
5 if (dfs(v)) match++;
6 }
7 return match;
8 }
9 boolean dfs(V v) {
10 v.used = true;
11 for (V u : v) {
12 V w = u.pair;
13 if (w == null || !w.used && dfs(w)) {
14 v.pair = u;
15 u.pair = v;
16 return true;
17 }
18 }
19 return false;
20 }
21 class V extends ArrayList<V> {
22 V pair;
23 boolean used;
24 void connect(V v) {
25 add(v);
26 v.add(this);
27 }
28 }

```

$O(\sqrt{V E})$

```

1 int hopcroftKarp(V[] vs) {
2   for (int match = 0;;) {
3     Queue<V> que = new LinkedList<V>();
4     for (V v : vs) v.level = -1;
5     for (V v : vs) if (v.pair == null) {
6       v.level = 0;
7       que.offer(v);
8     }
9     while (!que.isEmpty()) {
10      V v = que.poll();
11      for (V u : v) {
12        V w = u.pair;
13        if (w != null && w.level < 0) {
14          w.level = v.level + 1;
15          que.offer(w);
16        }
17      }
18    }
19    for (V v : vs) v.used = false;
20    int d = 0;
21    for (V v : vs) if (v.pair == null && dfs(v)) d++;
22    if (d == 0) return match;
23    match += d;
24  }
25 }
26 boolean dfs(V v) {
27   v.used = true;
28   for (V u : v) {
29     V w = u.pair;
30     if (w == null || !w.used && v.level < w.level && dfs(w)) {
31       v.pair = u;
32       u.pair = v;
33       return true;
34     }
35   }
36   return false;
37 }

```

3.12 3.12 Matching ổn định

Cách tìm stable matching tối ưu cho phía nam.

- Chọn một nam và cho anh ta cầu hôn người nữ ưu tiên cao nhất trong sổ chưa từng cầu hôn.
- Nếu người nữ thích nam này hơn bạn ghép hiện tại, hoặc chưa có bạn ghép, tạo cặp mới với nam này, và hủy cặp cũ.
- Người nam bị từ chối lập tức cầu hôn người nữ khác.

O(n²)

)

Trả về res[i] là bạn ghép của nam i.

orderM[i][j] là người nữ mà nam i thích thứ j.

preferW[i][j] là thứ hạng ưu tiên của nữ i đối với nam j.

```

1 int[] stableMatching(int[][] orderM, int[][] preferW) {
2   int n = orderM.length;
3   int[] pairM = new int[n], pairW = new int[n], p = new int[n];
4   fill(pairM, -1);
5   fill(pairW, -1);
6   for (int i = 0; i < n; i++) {
7     while (pairM[i] < 0) {
8       int w = orderM[i][p[i++]], m = pairW[w];
9       if (m == -1) {
10        pairM[i] = w;
11        pairW[w] = i;
12      } else if (preferW[w][i] < preferW[w][m]) {
13        pairM[m] = -1;

```

```

14 pairM[i] = w;
15 pairM[w] = i;
16 i = m;
17 }
18 }
19 }
20 return pairM;
21 }

```

3.13 3.13 Chu trình Euler

Đừng quên kiểm tra liên thông.

Với đồ thị vô hướng, cũng cần kiểm tra rev.

```

1 V[] eulerianWalk(V v) {
2 Stack<V> res = new Stack<V>(), stack = new Stack<V>();
3 stack.push(v);
4 while (!stack.isEmpty()) {
5 v = stack.pop();
6 while (v.p < v.size()) {
7 stack.push(v);
8 v = v.get(v.p++);
9 }
10 res.push(v);
11 }
12 return res.toArray(new V[0]);
13 }

```

3.14 3.14 LCA

Gọi $e[2*n-1]$ là thứ tự các nút được thăm khi DFS từ gốc, $v[2*n-1]$ là độ sâu tương ứng, và $a[n]$ là vị trí xuất hiện đầu tiên trong e của mỗi nút; đáp án là $e[RMQ_v([a[u], a[v]])]$.

3.15 3.15 2SAT

Với mỗi mệnh đề $(y \text{ or } z)$, tạo đồ thị có hai cạnh $\text{not } y \rightarrow z$ và $\text{not } z \rightarrow y$.

Phân rã SCC đồ thị này; việc một SCC chứa đồng thời x và $\text{not } x$ tương đương với công thức gốc không

↪ thỏa được

(unsatisfiable).

SCC chứa x đứng sau SCC chứa $\text{not } x$ trong thứ tự tô pô khi và chỉ khi x là true.

4 4 Hình học phẳng

4.1 4.1 Giao điểm và các phép tính liên quan

```

1 //線分と点の距離
2 double disSP(P p1, P p2, P q) {
3 if (p2.sub(p1).dot(q.sub(p1)) < EPS) return q.sub(p1).abs();
4 if (p1.sub(p2).dot(q.sub(p2)) < EPS) return q.sub(p2).abs();
5 return disLP(p1, p2, q);
6 }
7 //直線と点の距離
8 double disLP(P p1, P p2, P q) {
9 return abs(p2.sub(p1).det(q.sub(p1))) / p2.sub(p1).abs();
10 }
11 //線分と線分の交差判定
12 boolean crsSS(P p1, P p2, P q1, P q2) {
13 if (max(p1.x, p2.x) + EPS < min(q1.x, q2.x)) return false;
14 if (max(q1.x, q2.x) + EPS < min(p1.x, p2.x)) return false;

```

```

15 if (max(p1.y, p2.y) + EPS < min(q1.y, q2.y)) return false;
16 if (max(q1.y, q2.y) + EPS < min(p1.y, p2.y)) return false;
17 return sig(p2.sub(p1).det(q1.sub(p1))) * sig(p2.sub(p1).det(q2.sub(p1))) < EPS
18 && sig(q2.sub(q1).det(p1.sub(q1))) * sig(q2.sub(q1).det(p2.sub(q1))) < EPS;
19 }
20 //円と線分の交差判定
21 boolean crsCS(P c, double r, P p1, P p2) {
22 return disSP(p1, p2, c) < r + EPS &&
23 (r < c.sub(p1).abs() + EPS || r < c.sub(p2).abs() + EPS);
24 }
25 //円と円の交差判定
26 boolean crsCC(P c1, double r1, P c2, double r2) {
27 double dis = c1.sub(c2).abs();
28 return dis < r1 + r2 + EPS && abs(r1 - r2) < dis + EPS;
29 }
30 //四点の同一円周上判定 (一直線上は true)
31 boolean onCir(P p1, P p2, P p3, P p4) {
32 if (abs(p2.sub(p1).det(p3.sub(p1))) < EPS) return true;
33 P c = ccenter(p1, p2, p3);
34 return abs(c.sub(p1).abs2() - c.sub(p4).abs2()) < EPS;
35 }
36 //点から直線に下ろした垂線の足
37 P proj(P p1, P p2, P q) {
38 return p1.add(p2.sub(p1).mul(p2.sub(p1).dot(q.sub(p1)) / p2.sub(p1).abs2()));
39 }
40 //直線と直線の交点
41 P isLL(P p1, P p2, P q1, P q2) {
42 double d = q2.sub(q1).det(p2.sub(p1));
43 if (abs(d) < EPS) return null;
44 return p1.add(p2.sub(p1).mul(q2.sub(q1).det(q1.sub(p1)) / d));
45 }
46 //直線と円の交点 (p1 に近い順)
47 P[] isCL(P c, double r, P p1, P p2) {
48 double x = p1.sub(c).dot(p2.sub(p1));
49 double y = p2.sub(p1).abs2();
50 double d = x * x - y * (p1.sub(c).abs2() - r * r);
51 if (d < -EPS) return new P[0];
52 if (d < 0) d = 0;
53 P q1 = p1.sub(p2.sub(p1).mul(x / y));
54 P q2 = p2.sub(p1).mul(sqrt(d) / y);
55 return new P[]{q1.sub(q2), q1.add(q2)};
56 }
57 //二円の交点
58 P[] isCC(P c1, double r1, P c2, double r2) {
59 double x = c1.sub(c2).abs2();
60 double y = ((r1 * r1 - r2 * r2) / x + 1) / 2;
61 double d = r1 * r1 / x - y * y;
62 if (d < -EPS) return new P[0];
63 if (d < 0) d = 0;
64 P q1 = c1.add(c2.sub(c1).mul(y));
65 P q2 = c2.sub(c1).mul(sqrt(d)).rot90();
66 return new P[]{q1.sub(q2), q1.add(q2)};
67 }
68 //点 p をから引いた接線の接点
69 P[] tanCP(P c, double r, P p) {
70 double x = p.sub(c).abs2();
71 double d = x - r * r;
72 if (d < -EPS) return new P[0];
73 if (d < 0) d = 0;
74 P q1 = p.sub(c).mul(r * r / x);
75 P q2 = p.sub(c).mul(-r * sqrt(d) / x).rot90();
76 return new P[]{c.add(q1.sub(q2)), c.add(q1.add(q2))};
77 }
78 //二円の共通接線
79 P[][] tanCC(P c1, double r1, P c2, double r2) {
80 List<P[]> list = new ArrayList<P[]>();
81 if (abs(r1 - r2) < EPS) {
82 P dir = c2.sub(c1);
83 dir = dir.mul(r1 / dir.abs()).rot90();

```

```

84 list.add(new P[] {c1.add(dir), c2.add(dir)});
85 list.add(new P[] {c1.sub(dir), c2.sub(dir)});
86 } else {
87 P p = c1.mul(-r2).add(c2.mul(r1)).div(r1 - r2);
88 P[] ps = tanCP(c1, r1, p);
89 P[] qs = tanCP(c2, r2, p);
90 for (int i = 0; i < ps.length && i < qs.length; i++) {
91 list.add(new P[] {ps[i], qs[i]});
92 }
93 }
94 P p = c1.mul(r2).add(c2.mul(r1)).div(r1 + r2);
95 P[] ps = tanCP(c1, r1, p);
96 P[] qs = tanCP(c2, r2, p);
97 for (int i = 0; i < ps.length && i < qs.length; i++) {
98 list.add(new P[] {ps[i], qs[i]});
99 }
100 return list.toArray(new P[0] []);
101 }
102 // 2円の共通部分面積
103 double areaCC(P c1, double r1, P c2, double r2) {
104 double d = c1.sub(c2).abs();
105 if (r1 + r2 < d + EPS) return 0;
106 if (d < abs(r1 - r2) + EPS) {
107 double r = min(r1, r2);
108 return r * r * PI;
109 }
110 double x = (d * d + r1 * r1 - r2 * r2) / (2 * d);
111 double t1 = acos(x / r1);
112 double t2 = acos((d - x) / r2);
113 return r1 * r1 * t1 + r2 * r2 * t2 - d * r1 * sin(t1);
114 }
115 // 原点中心半径 r の円と原点と p1, p2 からなる三角形の共通部分面積
116 double areaCT(double r, P p1, P p2) {
117 P[] qs = isCL(0, r, p1, p2);
118 if (qs.length == 0) return r * r * rad(p1, p2) / 2;
119 boolean b1 = p1.abs() > r + EPS, b2 = p2.abs() > r + EPS;
120 if (b1 && b2) {
121 if (p1.sub(qs[0]).dot(p2.sub(qs[0])) < EPS &&
122 p1.sub(qs[1]).dot(p2.sub(qs[1])) < EPS) {
123 return (r * r * (rad(p1, p2) - rad(qs[0], qs[1])) + qs[0].det(qs[1])) / 2;
124 } else {
125 return r * r * rad(p1, p2) / 2;
126 }
127 } else if (b1) {
128 return (r * r * rad(p1, qs[0]) + qs[0].det(p2)) / 2;
129 } else if (b2) {
130 return (r * r * rad(qs[1], p2) + p1.det(qs[1])) / 2;
131 } else {
132 return p1.det(p2) / 2;
133 }
134 }

```

4.2 4.2 Bao lỗi

Tạo bao lỗi theo chiều ngược kim đồng hồ.

Không bao gồm điểm nằm trên cạnh.

Có thể chọn cách xử lý điểm trên cạnh bằng cách đổi dấu bất đẳng thức.

Nếu bao gồm điểm trên cạnh thì không được có điểm trùng.

```

1 P[] convexHull(P[] ps) {
2 int n = ps.length, k = 0;
3 if (n <= 1) return ps;
4 sort(ps);
5 P[] qs = new P[n * 2];
6 for (int i = 0; i < n; qs[k++] = ps[i++]) {
7 while (k > 1 && qs[k - 1].sub(qs[k - 2]).det(ps[i].sub(qs[k - 1])) < EPS) k--;

```

```

8 }
9 for (int i = n - 2, t = k; i >= 0; qs[k++] = ps[i--]) {
10 while (k > t && qs[k - 1].sub(qs[k - 2]).det(ps[i].sub(qs[k - 1])) < EPS) k--;
11 }
12 P[] res = new P[k - 1];
13 System.arraycopy(qs, 0, res, 0, k - 1);
14 return res;
15 }

```

4.3 Cắt đa giác lồi

$O(n)$

Cắt đa giác lồi bởi đường thẳng p_1p_2 và giữ phía bên trái.

```

1 P[] convexCut(P[] ps, P p1, P p2) {
2 int n = ps.length;
3 ArrayList<P> res = new ArrayList<P>();
4 for (int i = 0; i < n; i++) {
5 int d1 = sig(p2.sub(p1).det(ps[i].sub(p1)));
6 int d2 = sig(p2.sub(p1).det(ps[(i + 1) % n].sub(p1)));
7 if (d1 >= 0) res.add(ps[i]);
8 if (d1 * d2 < 0) res.add(isLL(p1, p2, ps[i], ps[(i + 1) % n]));
9 }
10 return res.toArray(new P[0]);
11 }

```

4.4 Arrangement của các đoạn thẳng

Lấy đầu mút và giao điểm của đoạn thẳng làm đỉnh; nếu hai điểm cùng nằm trên một đoạn và giữa chúng

↪ không có điểm khác, dùng khoảng cách giữa chúng làm trọng số để tạo cạnh của đồ thị.

$O(n^3)$

)

```

1 V[] segmentArrangement(P[] ps1, P[] ps2) {
2 int n = ps1.length;
3 TreeSet<V> set = new TreeSet<V>();
4 for (int i = 0; i < n; i++) {
5 set.add(new V(ps1[i]));
6 set.add(new V(ps2[i]));
7 for (int j = 0; j < i; j++) {
8 //線分交差判定のうち、端点が他の線分上の条件は抜くこと
9 if (crsSS(ps1[i], ps2[i], ps1[j], ps2[j])) {
10 set.add(new V(isLL(ps1[i], ps2[i], ps1[j], ps2[j])));
11 }
12 }
13 }
14 V[] vs = set.toArray(new V[0]);
15 for (int i = 0; i < n; i++) {
16 ArrayList<V> list = new ArrayList<V>();
17 for (V v : vs) {
18 if (crsSP(ps1[i], ps2[i], v.p)) list.add(v);
19 }
20 V[] us = list.toArray(new V[0]);
21 sort(us);
22 for (int j = 0; j + 1 < us.length; j++) {
23 us[j].connect(us[j + 1]);
24 }
25 }
26 return vs;
27 }

```

4.5 4.5 Kiểm tra điểm nằm trong/ngoài đa giác

$O(n)$

Trả về 1 nếu ở trong, 0 nếu nằm trên cạnh, và -1 nếu ở ngoài.

```
1 int contains(P[] ps, P q) {
2   int n = ps.length;
3   int res = -1;
4   for (int i = 0; i < n; i++) {
5     P a = ps[i].sub(q), b = ps[(i + 1) % n].sub(q);
6     if (a.y > b.y) {
7       P t = a; a = b; b = t;
8     }
9     if (a.y < EPS && b.y > EPS && a.det(b) > EPS) {
10      res = -res;
11    }
12    if (abs(a.det(b)) < EPS && a.dot(b) < EPS) return 0;
13  }
14  return res;
15 }
```

4.6 4.6 Khoảng cách từ đa giác lồi đến điểm bên ngoài

Chia mặt phẳng bằng các pháp tuyến của cạnh đa giác lồi, rồi dùng tìm kiếm nhị phân để xác định miền chứa điểm.

$O(\log n)$

Giả sử đa giác lồi được cho theo chiều ngược kim đồng hồ.

```
1 double disConvexP(P[] ps, P q) {
2   int n = ps.length;
3   int left = 0, right = n;
4   while (right - left > 1) {
5     int mid = (left + right) / 2;
6     if (in(ps[(left + n - 1) % n], ps[left], ps[mid], ps[(mid + 1) % n], q)) {
7       right = mid;
8     } else {
9       left = mid;
10    }
11  }
12  return disSP(ps[left], ps[right % n], q);
13 }
14 boolean in(P p1, P p2, P p3, P p4, P q) {
15   P o12 = p1.sub(p2).rot90();
16   P o23 = p2.sub(p3).rot90();
17   P o34 = p3.sub(p4).rot90();
18   return in(o12, o23, q.sub(p2)) || in(o23, o34, q.sub(p3))
19   || in(o23, p3.sub(p2), q.sub(p2)) && in(p2.sub(p3), o23, q.sub(p3));
20 }
21 boolean in(P p1, P p2, P q) {
22   return p1.det(q) > -EPS && p2.det(q) < EPS;
23 }
```

4.7 4.7 Đường kính của đa giác lồi

Dùng rotating calipers để tìm khoảng cách xa nhất giữa hai điểm trên đa giác lồi.

$O(n)$

```
1 double convexDiameter(P[] ps) {
2   int n = ps.length;
3   int is = 0, js = 0;
4   for (int i = 1; i < n; i++) {
5     if (ps[i].x > ps[js].x) js = i;
```



```

6 if (ps[i].x < ps[js].x) js = i;
7 }
8 double maxd = ps[is].sub(ps[js]).abs();
9 int i = is, j = js;
10 do {
11 if (ps[(i + 1) % n].sub(ps[i]).det(ps[(j + 1) % n].sub(ps[j])) >= 0) {
12 j = (j + 1) % n;
13 } else {
14 i = (i + 1) % n;
15 }
16 maxd = max(maxd, ps[i].sub(ps[j]).abs());
17 } while (i != is || j != js);
18 return maxd;
19 }

```

4.8 Tập công thức

4.8.1 Tâm nội tiếp tam giác

```

a
-
-
A + b
-
-
B + c
-
-
C
a + b + c

```

4.8.2 Tâm ngoại tiếp tam giác

```

-
-
A +
-
-
B -
-
-
BC •
-
-
CA
-
-
AB ×
-
-
BC
-
-
-

```

ABT
2

4.8.3 4.8.3 Trực tâm tam giác

-
→
H = 3
-
→
G = 2
-
→
O

4.8.4 4.8.4 Tâm bàng tiếp tam giác

-a
-
→
A + b
-
→
B + c
-
→
C
-a + b + c
Hai điểm còn lại cũng tương tự.

4.8.5 4.8.5 Bán kính đường tròn ngoại tiếp tam giác

R =
 $\frac{abc}{4S}$

4.8.6 4.8.6 Công thức Heron

$2s := a + b + c$
S =
 $\sqrt{s(s - a)(s - b)(s - c)}$

4.8.7 4.8.7 Công thức Pick

Với đa giác đơn có mọi đỉnh nằm trên điểm nguyên, nếu S là diện tích, B là số điểm nguyên trên
↪ biên, I là số điểm nguyên bên trong, thì:
S =
B
2
+ I - 1

5 5 Hình học không gian

5.1 5.1 Giao điểm và các phép tính liên quan

```
1 //直線と点の距離
2 double disLP(P p1, P p2, P q) {
3     return p2.sub(p1).det(q.sub(p1)).abs() / p2.sub(p1).abs();
4 }
5 //直線と直線の距離
6 double disLL(P p1, P p2, P q1, P q2) {
7     P p = q1.sub(p1);
8     P u = p2.sub(p1);
9     P v = q2.sub(q1);
10    double d = u.abs2() * v.abs2() - u.dot(v) * u.dot(v);
11    if (abs(d) < EPS) return disLP(q1, q2, p1);
12    double s = (p.dot(u) * v.abs2() - p.dot(v) * u.dot(v)) / d;
13    return disLP(q1, q2, p1.add(u.mul(s)));
14 }
15 //平面と直線の交点
16 P isFL(P p, P o, P q1, P q2) {
17     double a = o.dot(q2.sub(p));
18     double b = o.dot(q1.sub(p));
19     double d = a - b;
20     if (abs(d) < EPS) return null;
21     return q1.mul(a).sub(q2.mul(b)).div(d);
22 }
23 //平面と平面の交線
24 P[] isFF(P p1, P o1, P p2, P o2) {
25     P e = o1.det(o2);
26     P v = o1.det(e);
27     double d = o2.dot(v);
28     if (abs(d) < EPS) return null;
29     P q = p1.add(v.mul(o2.dot(p2.sub(p1)) / d));
30     return new P[]{q, q.add(e)};
31 }
```

5.2 5.2 Bao lồi không gian

Trả về các mặt sao cho nhìn từ bên ngoài thì theo chiều ngược kim đồng hồ.

Cần nhiều để không có bốn điểm đồng phẳng.

$O(n^2)$

)

```
1 P[][] convexHull(P[] ps) {
2     int n = ps.length;
3     //vs[i][j]=辺 i->j をこの向きに含む三角形が、まだ調べてない:0、残った:-1、取り除かれた:1
4     int[][] vs = new int[n][n];
5     List<int[]> crt = new ArrayList<int[]>();
6     crt.add(new int[]{0, 1, 2});
7     crt.add(new int[]{2, 1, 0});
8     for (int i = 3; i < n; i++) {
9         List<int[]> next = new ArrayList<int[]>();
10        for (int[] t : crt) {
11            int v = ps[t[1]].sub(ps[t[0]]).det(ps[t[2]].sub(ps[t[0]])).dot(
12                ps[i].sub(ps[t[0]])) < 0 ? -1 : 1;
13            if (v < 0) next.add(t);
14            for (int j = 0; j < 3; j++) {
15                if (vs[t[(j + 1) % 3]][t[j]] == 0) {
16                    vs[t[j]][t[(j + 1) % 3]] = v;
17                } else {
18                    if (vs[t[(j + 1) % 3]][t[j]] != v) {
19                        if (v > 0) next.add(new int[]{t[j], t[(j + 1) % 3], i});
20                        else next.add(new int[]{t[(j + 1) % 3], t[j], i});
21                    }
22                }
23            }
24        }
25        crt = next;
26    }
27    vs[t[(j + 1) % 3]][t[j]] = 0;
28 }
```

```

23 }
24 }
25 }
26 crt = next;
27 }
28 P[] [] pss = new P[crt.size()][3];
29 for (int i = 0; i < pss.length; i++) {
30 for (int j = 0; j < 3; j++) pss[i][j] = ps[crt.get(i)][j];
31 }
32 return pss;
33 }

```

5.3 5.3 Cắt đa diện lồi

$O(n \log n)$

Cắt đa diện lồi bằng mặt phẳng po và giữ phía sau.

Đa diện lồi phải được định hướng ngược kim đồng hồ khi nhìn từ ngoài.

```

1 P[] [] convexCut(P[] [] pss, P p, P o) {
2 ArrayList<P[]> res = new ArrayList<P[]>();
3 ArrayList<P> sec = new ArrayList<P>();
4 for (P[] ps : pss) {
5 int n = ps.length;
6 ArrayList<P> qs = new ArrayList<P>();
7 boolean dif = false;
8 for (int i = 0; i < n; i++) {
9 int d1 = sig(o.dot(ps[i].sub(p)));
10 int d2 = sig(o.dot(ps[(i + 1) % n].sub(p)));
11 if (d1 <= 0) qs.add(ps[i]);
12 if (d1 * d2 < 0) {
13 P q = isFL(p, o, ps[i], ps[(i + 1) % n]);
14 qs.add(q);
15 sec.add(q);
16 }
17 if (d1 == 0) sec.add(ps[i]);
18 else dif = true;
19 dif |= o.dot(ps[(i + 1) % n].sub(ps[i])).det(ps[(i + 2) % n].sub(ps[i])) < -EPS;
20 }
21 if (qs.size() > 0 && dif) res.add(qs.toArray(new P[0]));
22 }
23 //2D の convexHull に dot(o) をつけたもの
24 if (sec.size() > 0) res.add(convexHull2D(sec.toArray(new P[0]), o));
25 return res.toArray(new P[0][]);
26 }
27 //空間の初期化
28 P[] [] init() {
29 P[] [] pss = new P[6][4];
30 pss[0][0] = pss[1][0] = pss[2][0] = new P(-INF, -INF, -INF);
31 pss[0][3] = pss[1][1] = pss[5][2] = new P(-INF, -INF, INF);
32 pss[0][1] = pss[2][3] = pss[4][2] = new P(-INF, INF, -INF);
33 pss[0][2] = pss[5][3] = pss[4][1] = new P(-INF, INF, INF);
34 pss[1][3] = pss[2][1] = pss[3][2] = new P( INF, -INF, -INF);
35 pss[1][2] = pss[5][1] = pss[3][3] = new P( INF, -INF, INF);
36 pss[2][2] = pss[4][3] = pss[3][1] = new P( INF, INF, -INF);
37 pss[5][0] = pss[4][0] = pss[3][0] = new P( INF, INF, INF);
38 return pss;
39 }

```

5.4 5.4 Tập công thức

5.4.1 5.4.1 Công thức tứ diện Euler

Thể tích tứ diện có sáu cạnh dài a, b, c, d, e, f .

Ghi chú/công thức: $O = ABC$ $a = AB, b = BC, c = CA, d = OC, e = OA, f = OB$

$(12V)^2$
 $= a^2$
 d^2
 $(b^2$
 $+ c^2$
 $+ e^2$
 $+ f^2$
 $- a^2$
 $- d^2$
 $) + b^2$
 e^2
 $(c^2$
 $+ a^2$
 $+ f^2$
 $+ d^2$
 $- b^2$
 $- e^2$
 $) + c^2$
 f^2
 $(a^2$
 $+ b^2$
 $+ d^2$
 $+ e^2$
 $-$
 c^2
 $- f^2$
 $) - a^2$
 b^2
 c^2
 $- a^2$
 e^2
 f^2
 $- d^2$
 b^2
 f^2
 $- d^2$
 e^2
 c^2

5.5 Phép biến đổi

5.5.1 Biến đổi đối ngẫu

Phương pháp biến đổi.

Ghi chú/công thức: $d \cdot p = (p_1, p_2, \dots, p_d) \cdot d \cdot p^*$

Ghi chú/công thức: $x_d = p_1x_1 + p_2x_2 + \dots + p_{d-1}x_{d-1} - p_d$

Ghi chú/công thức:

Ghi chú/công thức: $d \cdot h : x_d = a_1x_1 + a_2x_2 + \dots + a_d \cdot d \cdot h^*$

Ghi chú/công thức: $= (a_1, a_2, \dots, a_{d-1}, -a_d)$

Tính chất.

Ghi chú/công thức:

Nói cách khác:

$p \in h \Leftrightarrow h^*$

$\in p^*$

Ghi chú/công thức: $p \rightarrow h \Leftrightarrow h^*$

Ghi chú/công thức: p^*

Ghi chú/công thức:

Envelope và bao lồi.

Ghi chú/công thức: $h \rightarrow H \rightarrow h^*$

Ghi chú/công thức: H^*

Ghi chú/công thức:

Ghi chú/công thức:

5.5.2 5.5.2 Biến đổi nâng/inversion 1

Phương pháp biến đổi.

Ghi chú/công thức: $p = (x, y) \rightarrow p'$

$= (x, y, x^2$

$+ y^2$

Ghi chú/công thức: $)$

Ghi chú/công thức: $C : (x - a)^2$

$+ (y - b)^2$

$= r^2$

Ghi chú/công thức: C'

$: z = a(x - a) + b(y - b) + r^2$

Ghi chú/công thức:

Tính chất.

Ghi chú/công thức:

Nói cách khác:

Ghi chú/công thức: $p \in C \Leftrightarrow p' \in C'$

Ghi chú/công thức: C'

Ghi chú/công thức:

Phân hoạch Delaunay và biểu đồ Voronoi.

Ghi chú/công thức: $P \rightarrow P'$

Ghi chú/công thức:

P'

Ghi chú/công thức: $z = 0 \rightarrow P \rightarrow P'^*$

Ghi chú/công thức:

Ghi chú/công thức: $z = 0 \rightarrow P$

5.5.3 5.5.3 Biến đổi inversion 2

Phương pháp biến đổi.

Ghi chú/công thức: $p = (r, \theta) \rightarrow p'$

$= (1$

Ghi chú/công thức: $r, \theta)$

Ghi chú/công thức:

Tính chất.

Ghi chú/công thức:

6 6 Toán học

6.1 6.1 Đếm tổ hợp

6.1.1 6.1.1 Số Catalan

$C(n) =$

$\frac{2n!}{n!(n+1)!}$

$n + 1$

Số cách tạo cây nhị phân với n nút là $C(n)$.

Ghi chú/công thức: $n \geq 0 \quad C(n) = \frac{1}{n+1} \binom{2n}{n}$

6.1.2 6.1.2 Tổ hợp có lặp

Tổng số cách chọn k phần tử từ n loại, trong đó loại i có $m[i]$ phần tử.

$x[i, j] = x[i - 1, j] + x[i, j - 1] - x[i - 1, j - m[i] - 1]$

6.1.3 6.1.3 Số phân hoạch

Số cách phân hoạch n thành k số nguyên không âm.

Ghi chú/công thức: $P(n - k, k) = P(n, n)$

$P[i, j] = P[i - 1, j] + P[i, j - i]$

Phân hoạch thành số lượng tùy ý các số tự nhiên có thể tính nhanh hơn.

$O(n^3)$

3

```
2 )
1 int partition(int n) {
2 int[] dp = new int[n + 1];
3 dp[0] = 1;
4 for (int i = 1; i <= n; i++) {
5 for (int j = 1, r = 1; i - (3 * j * j - j) / 2 >= 0; j++, r *= -1) {
6 dp[i] += dp[i - (3 * j * j - j) / 2] * r;
7 if (i - (3 * j * j + j) / 2 >= 0) {
8 dp[i] += dp[i - (3 * j * j + j) / 2] * r;
9 }
10 }
11 }
12 return dp[n];
13 }
```

6.1.4 6.1.4 Số Bell

Tổng số cách chia n phần tử phân biệt thành các nhóm.

$B[n] =$

$\sum_{i=0}^{n-1} C[n - 1, i] B[i]$

6.1.5 6.1.5 Số Stirling loại một

Số cách chia n phần tử phân biệt thành k chu trình không rỗng.

Đồng thời là hệ số khi biểu diễn lũy thừa giai thừa tăng bằng tổng các lũy thừa thường.

$S[n, k] = (n - 1)S[n - 1, k] + S[n - 1, k - 1]$

6.1.6 6.1.6 Số Stirling loại hai

Số cách chia n phần tử phân biệt thành k nhóm không rỗng.

Đồng thời là hệ số khi biểu diễn lũy thừa thường bằng tổng các lũy thừa giai thừa giảm.

$$B[n] =$$

$$\sum_n$$

Ghi chú/công thức: $i=1 \dots n$ $S[n, i]$

$$S[n, k] = kS[n-1, k] + S[n-1, k-1]$$

Khi chia thành các nhóm có ít nhất r phần tử:

$$S[n, k] = kS[n-1, k] + C[n-1, r-1]S[n-r, k-1]$$

ta có công thức sau.

6.1.7 6.1.7 Số Euler

Số hoán vị trên $\{1, \dots, n\}$ có đúng k lần tăng.

$$E[n, k] = (k+1)E[n-1, k] + (n-k)E[n-1, k-1]$$

6.1.8 6.1.8 Nguyên lý bao hàm - loại trừ

$$U$$

$$P \in A$$

$$P =$$

$$\sum$$

$$\emptyset \subset S \subseteq A$$

$$(-1)^{|S|} - 1$$

$$n$$

$$P \in S$$

$$P$$

$$g(A) =$$

$$\sum$$

$$S \subseteq A$$

$$f(S) \Leftrightarrow f(A) =$$

$$\sum$$

$$S \subseteq A$$

$$(-1)^{|A|} - |S|$$

$$g(S)$$

6.1.9 6.1.9 Hàm Mobius

$$\mu(n) =$$

$$\{$$

$\emptyset$ (khi n có thừa số chính phương)

$$(-1)^k$$

Ghi chú/công thức: $(n \mid k)$

$$g(n) =$$

$$\sum$$

$$d \mid n$$

$$f(d) \Leftrightarrow f(n) =$$

$$\sum$$

$$d \mid n$$

$$\mu(d)g($$

$$n$$

$$d$$

$$)$$

$$g(x) = \sum_{n=1}^{\infty} \frac{f(x)}{x^n} \Leftrightarrow f(x) = \sum_{n=1}^{\infty} \mu(n)g\left(\frac{x}{n}\right)$$

6.1.10 6.1.10 Bổ đề Burnside, tên cũ

$$|X/G| = \frac{1}{|G|} \sum_{g \in G} |Xg|$$

6.2 6.2 Lý thuyết số

6.2.1 6.2.1 Tập công thức

Định lý nhỏ Fermat.

$$ap$$

$$\equiv a \pmod{p}$$

Ghi chú/công thức: $(a, p) = 1 \quad ap-1$

$$\equiv 1 \pmod{p}$$

Ghi chú/công thức: $a-1$

$$\equiv ap-2$$

$$\pmod{p}$$

Hàm Euler phi.

$\phi(n)$ là số số tự nhiên từ 1 đến n nguyên tố cùng nhau với n.

$$\phi(n) = n$$

$$\prod$$

$$p|n$$

$$(1 -$$

$$1$$

$$p$$

$$)$$

Ghi chú/công thức: $(a, n) = 1 \quad a\phi(n)$

$$\equiv 1 \pmod{n}$$

Bậc.

$$ax$$

Ghi chú/công thức: $\equiv 1 \pmod{m} \quad x \quad a \pmod{m}$

Ghi chú/công thức: $a \pmod{m} \quad \phi(m)$

a được gọi là căn nguyên thủy modulo m.

Ghi chú/công thức: $p \quad \phi(p - 1)$

xd

Ghi chú/công thức: $\equiv 1 \pmod{p}$

Ghi chú/công thức: $d \mid (p - 1) \iff (d) \mid \phi(d) \iff d$

Định lý Wilson.

$(p - 1)! \equiv -1 \pmod{p}$

Thặng dư bậc hai.

Với số nguyên tố lẻ p :

x^2

Ghi chú/công thức: $\equiv a \pmod{p} \iff a$

$p-1$

$2 \equiv 1 \pmod{p}$

Cây Stern-Brocot.

Đây là cây tìm kiếm nhị phân có các khoảng hữu tỉ làm đỉnh, được định nghĩa như sau.

Gốc là:

(

0

1

,

1

0

)

Với đỉnh

(a

b

,

c

d

)

các con là

(

a

b

,

a + c

b + d

)

Ghi chú/công thức:

(

a + c

b + d

,

c

d

)

Cây này có các tính chất sau.

Các đầu mút của mỗi đỉnh là phân số tối giản.

Mọi phân số tối giản xuất hiện đúng một lần.

Theo độ sâu của cây, tử số và mẫu số tăng đơn điệu.

Với đỉnh

(a

b

,

c

d

```

)
với định đồ,
 $ad - bc = 1$ 
Công thức Stirling.
 $n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$ 
 $n! \equiv n! \pmod{p}$ 
Ghi chú/công thức:  $n! \equiv k + (n - k) \pmod{p}$ 
Ghi chú/công thức:  $n = \sum_{i=1}^n n_i p_i$ ,  $k = \sum_{i=1}^n k_i p_i$ 
Ghi chú/công thức:
 $n! \equiv \prod_{i=1}^n i! \pmod{p}$ 

```

6.2.2 Giai thừa modulo

Với số nguyên tố p , viết $n! = a \cdot p^k$.
 Tìm giá trị a modulo p .
 $O(p \log n)$

```

1 int modFact(int n, int p) {
2   int res = 1;
3   while (n > 0) {
4     for (int i = 1, m = n % p; i <= m; i++) res = (res * i) % p;
5     if ((n /= p) % 2 > 0) res = p - res;
6   }
7   return res;
8 }

```

6.2.3 Thuật toán Euclid mở rộng

Tìm một bộ $\{a, b, c = \gcd(x, y)\}$ sao cho $ax + by = \gcd(x, y)$.
 Ghi chú/công thức: $(a, b) \rightarrow (a + dy, c, b - dx, c)$

```

1 int[] exgcd(int x, int y) {
2   int a0 = 1, a1 = 0, b0 = 0, b1 = 1, t;
3   while (y != 0) {
4     t = a0 - x / y * a1; a0 = a1; a1 = t;
5     t = b0 - x / y * b1; b0 = b1; b1 = t;
6     t = x % y; x = y; y = t;
7   }
8   if (x < 0) {
9     a0 = -a0; b0 = -b0; x = -x;
10  }
11  return new int[] {a0, b0, x};
12 }

```

6.2.4 6.2.4 Hệ đồng dư tuyến tính

Trả về nghiệm và modulo $\{x, m\}$ của hệ đồng dư tuyến tính $Ax \equiv B \pmod{M}$.
Nếu không tồn tại nghiệm thì trả về null.

```
1 BigInt[] congruence(BigInt[] A, BigInt[] B, BigInt[] M) {
2   BigInt x = 0, m = 1;
3   for (int i = 0; i < A.length; i++) {
4     BigInt a = A[i] * m, b = B[i] - A[i] * x, d = a.gcd(M[i]);
5     if (b % d != 0) return null;
6     x += m * (b / d * (a / d).modInv(M[i] / d) % (M[i] / d));
7     m *= M[i] / d;
8   }
9   return new BigInt[] {x % m, m};
10 }
```

6.2.5 6.2.5 Logarit rời rạc

ax

Ghi chú/công thức: $\equiv b \pmod{m}$ x

Ghi chú/công thức: -1

```
1 long modLog(long a, long b, long m) {
2   if (b % exgcd(a, m)[2] != 0) return -1;
3   if (m == 0) return 0;
4   long n = (long)sqrt(m) + 1;
5   Map<Long, Long> map = new HashMap<Long, Long>();
6   long an = 1;
7   for (long j = 0; j < n; j++) {
8     if (!map.containsKey(an)) map.put(an, j);
9     an = an * a % m;
10  }
11  long ain = 1, res = Long.MAX_VALUE;
12  for (long i = 0; i < n; i++) {
13    long[] is = congruence(ain, b, m);
14    for (long aj = is[0]; aj < m; aj += is[1]) if (map.containsKey(aj)) {
15      long j = map.get(aj);
16      res = min(res, i * n + j);
17    }
18    if (res < Long.MAX_VALUE) return res;
19    ain = ain * an % m;
20  }
21  return -1;
22 }
```

6.2.6 6.2.6 Giải đồng dư modulo hợp số

Dạng tuyến tính.

$ax + by = n$ có nghiệm khi và chỉ khi $\gcd(a, b)$ chia n .

$ka \equiv kb \pmod{m} \Leftrightarrow a \equiv b \pmod{m}$

m

(k, m)

)

$kx \equiv 1 \pmod{m}$ có $\gcd(k, m)$ nghiệm khi $\gcd(k, m)$ chia 1.

Đa thức.

Muốn tìm nghiệm của $f(x) \equiv 0 \pmod{m}$.

$m =$

k

Π

$i=1$

$p \mid a_i$
 Ghi chú/công thức: i
 Ghi chú/công thức: $f(x) \equiv 0 \pmod{p}$
 Ghi chú/công thức: $i \in \{x_i\}_k$
 Ghi chú/công thức: $i=1$
 Một nghiệm được xác định.
 $f(x) \equiv 0 \pmod{p}$
 Ghi chú/công thức: i
 $f(x) \equiv 0 \pmod{p-1}$
 Ghi chú/công thức: $i \in \{x_i\}$
 Ghi chú/công thức:
 f'
 $(x'$
 Ghi chú/công thức: $i) \equiv 0 \pmod{p} \quad f(x'$
 $) \equiv 0 \pmod{p}$
 $) \equiv x'$
 $+ d p a - 1$
 $(d = 0, \dots, p - 1)$
 f'
 $(x'$
 $) \equiv 0 \pmod{p} \equiv x'$
 $-$
 $f(x'$
 $)$
 $f'(x')$

6.3 Đại số tuyến tính

6.3.1 Tập công thức

Công thức Cramer.
 Với $Ax=b$, gọi A_i là ma trận thay cột i của A bằng b .
 $x_i = \frac{|A_i|}{|A|}$
 Hệ tuyến tính ba ẩn nên giải bằng cách này.
 Biến đổi đối ngẫu.
 $\max\{cx \mid Ax \leq b, x \geq 0\} = \min\{yb \mid yA \geq c, y \geq 0\}$
 $\max\{cx \mid Ax = b, x \geq 0\} = \min\{yb \mid yA \geq c\}$
 $\max\{cx \mid Ax = b\} = \min\{yb \mid yA = c\}$

6.3.2 Hệ phương trình

Ghi chú/công thức: $Ax = 0 \quad X = \{x_0 + \sum$
 $\sum$
 Ghi chú/công thức: $a_i x_i\} \quad O(nm)$

```

1 double[][] solutionSpace(double[][] A, double[] b) {
2   int n = A.length, m = A[0].length;
3   double[][] a = new double[n][m + 1];
4   for (int i = 0; i < n; i++) {
5     System.arraycopy(A[i], 0, a[i], 0, m);
6     a[i][m] = b[i];
7   }
8   int[] id = new int[n + 1]; //単位行列の成分が (i, id[i]) になる
9   fill(id, -1);

```

```

10 int pi = 0; //ラングに相当
11 for (int pj = 0; pi < n && pj < m; pj++) {
12   for (int i = pi + 1; i < n; i++) {
13     if (abs(a[i][pj]) > abs(a[pi][pj])) {
14       double t = a[i]; a[i] = a[pi]; a[pi] = t;
15     }
16   }
17   if (abs(a[pi][pj]) < EPS) continue;
18   double inv = 1 / a[pi][pj];
19   for (int j = 0; j <= m; j++) a[pi][j] *= inv;
20   for (int i = 0; i < n; i++) if (i != pi) {
21     double d = a[i][pj];
22     for (int j = 0; j <= m; j++) a[i][j] -= d * a[pi][j];
23   }
24   id[pi++] = pj;
25 }
26 for (int i = pi; i < n; i++) if (abs(a[i][m]) > EPS) return null;
27 double[][] X = new double[1 + m - pi][m];
28 for (int j = 0, k = 0; j < m; j++) {
29   if (id[k] == j) X[0][j] = a[k++][m];
30   else {
31     for (int i = 0; i < k; i++) X[1 + j - k][id[i]] = -a[i][j];
32     X[1 + j - k][j] = 1;
33   }
34 }
35 return X;
36 }

```

Ghi chú/công thức: $\blacksquare$ Toeplitz $A \quad Ax = b \quad O(n^2)$

$A[i, j] = A[i - j + \text{ZERO}]$
 $A[0, 0] \neq 0$

```

1 double[] levinson(double[] A, double[] b) {
2   int n = (A.length + 1) / 2, ZERO = n - 1;
3   double[] x = new double[n], fs = new double[n], bs = new double[n];
4   fs[0] = bs[0] = 1 / A[ZERO];
5   x[0] = b[0] / A[ZERO];
6   for (int i = 1; i < n; i++) {
7     double ef = 0, eb = 0, ex = 0;
8     for (int j = 0; j < i; j++) {
9       ef += fs[j] * A[i - j + ZERO];
10      eb += bs[j] * A[-1 - j + ZERO];
11      ex += x[j] * A[i - j + ZERO];
12    }
13    if (abs(ef * eb - 1) < EPS) return null;
14    for (int j = i; j >= 0; j--) {
15      double af = (j < i ? fs[j] : 0), ab = (j != 0 ? bs[j - 1] : 0);
16      fs[j] = af / (1 - ef * eb) - ef * ab / (1 - ef * eb);
17      bs[j] = ab / (1 - ef * eb) - eb * af / (1 - ef * eb);
18    }
19    for (int j = 0; j <= i; j++) x[j] += (b[i] - ex) * bs[j];
20  }
21  return x;
22 }

```

6.3.3 Phương pháp đơn hình

Giải $\max\{cx \mid Ax \leq b, x \geq 0\}$.

Nếu không tồn tại nghiệm thì trả về null.

```

1 double[] simplex(double[][] A, double[] b, double[] c) {
2   int n = A.length, m = A[0].length + 1, r = n, s = m - 1;
3   double[][] D = new double[n + 2][m + 1];
4   int[] ix = new int[n + m];

```

```

5 for (int i = 0; i < n + m; i++) ix[i] = i;
6 for (int i = 0; i < n; i++) {
7   for (int j = 0; j < m - 1; j++) D[i][j] = -A[i][j];
8   D[i][m - 1] = 1;
9   D[i][m] = b[i];
10  if (D[r][m] > D[i][m]) r = i;
11 }
12 for (int j = 0; j < m - 1; j++) D[n][j] = c[j];
13 D[n + 1][m - 1] = -1;
14 for (double d;;) {
15   if (r < n) {
16     int t = ix[s]; ix[s] = ix[r + m]; ix[r + m] = t;
17     D[r][s] = 1.0 / D[r][s];
18     for (int j = 0; j <= m; j++) if (j != s) D[r][j] *= -D[r][s];
19     for (int i = 0; i <= n + 1; i++) if (i != r) {
20       for (int j = 0; j <= m; j++) if (j != s) D[i][j] += D[r][j] * D[i][s];
21       D[i][s] *= D[r][s];
22     }
23   }
24   r = -1; s = -1;
25   for (int j = 0; j < m; j++) if (s < 0 || ix[s] > ix[j]) {
26     if (D[n + 1][j] > EPS || D[n + 1][j] > -EPS && D[n][j] > EPS) s = j;
27   }
28   if (s < 0) break;
29   for (int i = 0; i < n; i++) if (D[i][s] < -EPS) {
30     if (r < 0 || (d = D[r][m] / D[r][s] - D[i][m] / D[i][s]) < -EPS
31     || d < EPS && ix[r + m] > ix[i + m]) r = i;
32   }
33   if (r < 0) return null; //非有界
34 }
35 if (D[n + 1][m] < -EPS) return null; //実行不能
36 double[] x = new double[m - 1];
37 for (int i = m; i < n + m; i++) if (ix[i] < m - 1) x[ix[i]] = D[i - m][m];
38 return x; //値は D[n][m]
39 }

```

6.3.4 6.3.4 Công thức truy hồi

Ghi chú/công thức: $a_{i+n} =$

$\sum$

Ghi chú/công thức: $k_j a_{i+j} + d \quad a_m =$

$\sum$

Ghi chú/công thức: $c_i a_i + cnd \quad \{c_i\}$

$O(n^2)$

$\log m$

Ghi chú/công thức: $\{k_i\} \quad FFT \quad O(n \log n \log m)$

```

1 double[] recFormula(double[] k, long m) {
2   int n = k.length;
3   double[] c = new double[n + 1];
4   if (m < n) c[(int)m] = 1;
5   else {
6     double[] b = recFormula(k, m >>> 1);
7     double[] a = new double[n * 2];
8     int s = (int)(m & 1);
9     for (int i = 0; i < n; i++) {
10      for (int j = 0; j < n; j++) {
11        a[i + j + s] += b[i] * b[j];
12      }
13      c[n] += b[i];
14    }
15    c[n] = (c[n] + 1) * b[n];
16    for (int i = n * 2 - 1; i >= n; i--) {
17      for (int j = 0; j < n; j++) {
18        a[i - n + j] += k[j] * a[i];

```

```

19 }
20 c[n] += a[i];
21 }
22 System.arraycopy(a, 0, c, 0, n);
23 }
24 return c;
25 }

```

6.3.5 6.3.5 Nghiệm nguyên

$O(n^5)$

Ghi chú/công thức:)

Nếu không có nghiệm trả về null; nếu có nhiều nghiệm, trả về một nghiệm.

```

1 BigInteger[] intSolve(BigInteger[] A, BigInteger[] b) {
2 int n = A.length, m = A[0].length;
3 BigInteger[] a = new BigInteger[n][m + 1];
4 for (int i = 0; i < n; i++) {
5 for (int j = 0; j < m; j++) a[i][j] = A[i][j];
6 a[i][m] = b[i];
7 }
8 Stack<Trans> trans = new Stack<Trans>();
9 for (int p = 0; p < m && p < n; p++) {
10 for (;;) {
11 int pi = p, pj = p;
12 for (int i = p; i < n; i++) {
13 for (int j = p; j < m; j++) {
14 if (a[i][j].signum() != 0 && (a[pi][pj].signum() == 0
15 || a[pi][pj].abs().compareTo(a[i][j].abs()) > 0)) {
16 pi = i;
17 pj = j;
18 }
19 }
20 }
21 swap(a, pi, p);
22 for (int i = p; i < n; i++) swap(a[i], pj, p);
23 if (pj != p) trans.push(new Trans(0, pj, p, null));
24 if (a[p][p].signum() == 0) break;
25 boolean end = true;
26 for (int i = p + 1; i < n; i++) {
27 BigInteger d = a[i][p].divide(a[p][p]);
28 end = end && a[i][p].signum() == 0;
29 for (int j = p; j <= m; j++) {
30 a[i][j] = a[i][j].subtract(a[p][j].multiply(d));
31 }
32 }
33 for (int j = p + 1; j < m; j++) {
34 BigInteger d = a[p][j].divide(a[p][p]);
35 end = end && a[p][j].signum() == 0;
36 trans.push(new Trans(1, j, p, d));
37 for (int i = p; i < n; i++) {
38 a[i][j] = a[i][j].subtract(a[i][p].multiply(d));
39 }
40 }
41 if (end) break;
42 }
43 }
44 BigInteger[] res = new BigInteger[m];
45 fill(res, ZERO);
46 for (int i = 0; i < m && i < n; i++) {
47 if (a[i][i].signum() < 0) {
48 a[i][i] = a[i][i].negate();
49 a[i][m] = a[i][m].negate();
50 }
51 if (a[i][i].signum() == 0) {
52 if (a[i][m].signum() != 0) return null;

```



```

53 } else if (a[i][m].mod(a[i][i]).signum() == 0) {
54 res[i] = a[i][m].divide(a[i][i]);
55 } else return null;
56 }
57 for (int i = min(m, n); i < n; i++) if (a[i][m].signum() != 0) return null;
58 while (!trans.isEmpty()) {
59 Trans t = trans.pop();
60 if (t.type == 0) swap(res, t.a, t.b);
61 else res[t.b] = res[t.b].subtract(res[t.a].multiply(t.c));
62 }
63 return res;
64 }

```

6.4 6.4 Giải tích

6.4.1 6.4.1 Biến đổi Fourier nhanh

■ $O(n \log n)$

Độ dài phải là lũy thừa của 2.

Dùng sign=1 cho biến đổi thuận, sign=-1 cho biến đổi ngược.

Khi biến đổi ngược, nhân kết quả với $1/n$.

```

1 void fft(int sign, double[] real, double[] imag) {
2 int n = real.length, d = Integer.numberOfLeadingZeros(n) + 1;
3 double theta = sign * 2 * PI / n;
4 for (int m = n; m >= 2; m >>= 1, theta *= 2) {
5 for (int i = 0, mh = m >> 1; i < mh; i++) {
6 double wr = cos(i * theta), wi = sin(i * theta);
7 for (int j = i; j < n; j += m) {
8 int k = j + mh;
9 double xr = real[j] - real[k], xi = imag[j] - imag[k];
10 real[j] += real[k];
11 imag[j] += imag[k];
12 real[k] = wr * xr - wi * xi;
13 imag[k] = wr * xi + wi * xr;
14 }
15 }
16 }
17 for (int i = 0; i < n; i++) {
18 int j = Integer.reverse(i) >>> d;
19 if (j < i) {
20 double tr = real[i]; real[i] = real[j]; real[j] = tr;
21 double ti = imag[i]; imag[i] = imag[j]; imag[j] = ti;
22 }
23 }
24 }

```

6.4.2 6.4.2 Khai triển Taylor

```

ex
=
∞
Σ
n=0
xn
n!
log(1 + x) =
∞
Σ
n=1
(-1)n+1

```

$$\begin{aligned} & n \\ & x^n \\ & (|x| < 1) \\ \sin x = & \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1)!} x^{2n+1} \\ \cos x = & \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n)!} x^{2n} \\ \arcsin x = & \sum_{n=0}^{\infty} \frac{(2n)!}{4n(n!)2(2n+1)} x^{2n+1} \end{aligned}$$

($|x| < 1$)

6.4.3 Tích phân số

Công thức Simpson.

Tích phân số bằng cách xấp xỉ $f(x)$ bởi hàm bậc hai.

$$\begin{aligned} \int_a^b f(x) dx \approx & h/3 * [f(x_0) \\ & + 2 * \sum_{j=1}^{n/2-1} f(x_{2j}) \\ & + 4 * \sum_{j=1}^{n/2} f(x_{2j-1}) \\ & + f(x_n)]. \end{aligned}$$

Ở đây n là số chẵn, $h=(b-a)/n$ và $x_i=a+ih$.

Ghi chú/công thức: $h \rightarrow 0(h^4)$

)

7 Chuỗi

7.1 KMP

Tìm mẫu p trong văn bản t .

$fail[i]$ là độ dài lớn nhất L trong $[0, i)$ sao cho prefix và suffix trùng nhau.

Ngoài ra, $n-fail[n]$ biểu diễn chu kỳ của chuỗi.

Ghi chú/công thức: $s \leftarrow rev(s) \quad s[i]=n-1-fail[i]$

Ghi chú/công thức: $i-j$

```
1 class KMP {
2   int m;
3   char[] p;
4   int[] fail;
5   KMP(char[] p) {
6     m = p.length;
```

```

7 this.p = p;
8 fail = new int[m + 1];
9 int crt = fail[0] = -1;
10 for (int i = 1; i <= m; i++) {
11 while (crt >= 0 && p[crt] != p[i - 1]) crt = fail[crt];
12 fail[i] = ++crt;
13 }
14 }
15 int searchFrom(char[] t) {
16 int n = t.length, count = 0;
17 for (int i = 0, j = 0; i < n; i++) {
18 while (j >= 0 && t[i] != p[j]) j = fail[j];
19 if (++j == m) {
20 count++;
21 j = fail[j];
22 }
23 }
24 return count;
25 }
26 }

```

7.2 7.2 AhoCorasick

Tìm nhiều mẫu ps trong văn bản t.

```

1 class AhoCorasick {
2 int m;
3 Node root;
4 AhoCorasick(char[] [] ps) {
5 m = ps.length;
6 root = new Node();
7 for (int i = 0; i < m; i++) {
8 Node t = root;
9 for (char c : ps[i]) {
10 if (!t.cs.containsKey(c)) t.cs.put(c, new Node());
11 t = t.cs.get(c);
12 }
13 t.accept.add(i);
14 }
15 Queue<Node> que = new LinkedList<Node>();
16 que.offer(root);
17 while (!que.isEmpty()) {
18 Node t = que.poll();
19 for (Map.Entry<Character, Node> e : t.cs.entrySet()) {
20 char c = e.getKey();
21 Node u = e.getValue();
22 que.offer(u);
23 Node r = t.fail;
24 while (r != null && !r.cs.containsKey(c)) r = r.fail;
25 if (r == null) u.fail = root;
26 else u.fail = r.cs.get(c);
27 u.accept.addAll(u.fail.accept);
28 }
29 }
30 }
31 int[] searchFrom(char[] t) {
32 int n = t.length;
33 int[] count = new int[m];
34 Node u = root;
35 for (int i = 0; i < n; i++) {
36 while (u != null && !u.cs.containsKey(t[i])) u = u.fail;
37 if (u == null) u = root;
38 else u = u.cs.get(t[i]);
39 for (int j : u.accept) count[j]++;
40 }
41 return count;
42 }

```

```

43 class Node {
44 Map<Character, Node> cs = new TreeMap<Character, Node>();
45 List<Integer> accept = new ArrayList<Integer>();
46 Node fail;
47 }
48 }

```

7.3 7.3 SuffixArray

$O(n \log n)$

```

1 class SuffixArray {
2 int n;
3 char[] cs;
4 int[] si, is; //ソート後と元のインデックスの対応
5 SuffixArray(char[] t) {
6 n = t.length;
7 cs = new char[n + 1];
8 System.arraycopy(t, 0, cs, 0, n);
9 cs[n] = 0; //番兵 (他の全ての値より小さくすること)
10 is = new int[n + 1];
11 for (int i = 0; i <= n; i++) is[i] = cs[i];
12 si = indexSort(is);
13 int[] a = new int[n + 1], b = new int[n + 1];
14 for (int h = 0; ; ) {
15 for (int i = 0; i < n; i++) {
16 int x = si[i + 1], y = si[i];
17 b[i + 1] = b[i];
18 if (is[x] > is[y] || is[x + h] > is[y + h]) b[i + 1]++;
19 }
20 for (int i = 0; i <= n; i++) is[si[i]] = b[i];
21 if (b[n] == n) break;
22 h = max(1, h << 1);
23 for (int k = h; k >= 0; k -= h) {
24 fill(b, 0);
25 b[0] = k;
26 for (int i = k; i <= n; i++) b[is[i]]++;
27 for (int i = 0; i < n; i++) b[i + 1] += b[i];
28 for (int i = n; i >= 0; i--) {
29 a[--b[si[i] + k > n ? 0 : is[si[i] + k]]] = si[i];
30 }
31 int[] tmp = si; si = a; a = tmp;
32 }
33 }
34 }
35 int[] hs; //hs[i] := ソート後の suffix の i と i+1 の LCP
36 void buildHs() {
37 hs = new int[n + 1];
38 for (int i = 0, h = 0; i < n; i++) {
39 for (int j = si[is[i] - 1]; cs[i + h] == cs[j + h]; h++);
40 hs[is[i] - 1] = h;
41 if (h > 0) h--;
42 }
43 }
44 RMQ rmq;
45 void buildRMQ() {
46 rmq = new RMQ(hs); //値を返す版
47 }
48 //位置 i, j から始まる部分列の LCP を求める
49 int getLCP(int i, int j) {
50 if (i == j) return n - i;
51 return rmq.query(min(is[i], is[j]), max(is[i], is[j]));
52 }
53 }

```

7.4 7.4 Palindrome

Tính chất palindrome: nếu [a,b), [a,c), [b,d) đều là palindrome thì [c,d) cũng là palindrome.

Tính độ dài palindrome trong $O(n)$.

len[i]: độ dài palindrome có tâm tại vị trí i/2.

```
1 int[] palindrome(char[] cs) {
2 int n = cs.length;
3 int[] len = new int[n * 2];
4 for (int i = 0, j = 0, k; i < n * 2; i += k, j = max(j - k, 0)) {
5 while (i - j >= 0 && i + j + 1 < n * 2
6 && cs[(i - j) / 2] == cs[(i + j + 1) / 2]) j++;
7 len[i] = j;
8 for (k = 1; i - k >= 0 && j - k >= 0 && len[i - k] != j - k; k++) {
9 len[i + k] = min(len[i - k], j - k);
10 }
11 }
12 return len;
13 }
```

8 8 Khác

8.1 8.1 Matroid

8.1.1 8.1.1 Hệ tiên đề độc lập

(M1) $\emptyset \in F$
(M2) $X \subseteq Y \in F \Rightarrow X \in F$
(M3) $X, Y \in F, |X| > |Y| \Rightarrow \exists x \in X \setminus Y. Y \cup \{x\} \in F$
(M3')

Ghi chú/công thức:) $X \subseteq E$

Mọi cơ sở của X đều có cùng số phần tử.

8.1.2 8.1.2 Hệ tiên đề hạng

(R1) $r(X) \leq |X|$
(R2) $X \subseteq Y \Rightarrow r(X) \leq r(Y)$
(R3) $r(X \cup Y) + r(X \cap Y) \leq r(X) + r(Y)$

8.1.3 8.1.3 Giao matroid

Giao của hai matroid có thể không phải matroid, nhưng thuật toán sau tìm được tập độc lập lớn nhất.

Tìm được tập độc lập.

1. Khởi tạo $X = \text{empty}$.

2. Lấy $E \text{ union } \{s, t\}$ làm tập đỉnh; dựng cạnh như sau rồi tìm đường ngắn nhất từ s đến t.

(s,y): thêm y vào X vẫn độc lập trong F1.

(y,t): thêm y vào X vẫn độc lập trong F2.

(x,y): thêm y vào X làm mất độc lập trong F1, nhưng
bỏ x thì độc lập trở lại.

(y,x): thêm y vào X làm mất độc lập trong F2, nhưng
bỏ x thì độc lập trở lại.

3. Nếu không tồn tại đường ngắn nhất thì X là tối đại/lớn nhất.

Nếu có, cập nhật $X := X \text{ union } \{y_0, \dots, y_m\} \text{ minus } \{x_1, \dots, x_m\}$, rồi quay lại bước 2.

8.1.4 8.1.4 Phân hoạch matroid

Ghi chú/công thức: $X = X_1 \cup \dots \cup X_k \quad X_i \in F_i \quad X \in F$

Ghi chú/công thức: E'

Ghi chú/công thức: $= E \times \{1, \dots, k\} \quad 2$

$F_1 = \{Q \subseteq E'$

Ghi chú/công thức: $: i \quad Q_i \in F_i\}$

$F_2 = \{Q \subseteq E'$

Ghi chú/công thức: $: i \neq j \quad Q_i \cap Q_j = \emptyset\}$

Ghi chú/công thức: $Q_i = \{e \in E'$

$: (e, i) \in Q\}$

8.2 8.2 Lý thuyết trò chơi tổ hợp

8.2.1 8.2.1 Xác định trạng thái thắng

Thắng khi và chỉ khi tồn tại nước đi đến trạng thái thua.

Thua khi và chỉ khi mọi nước đi đều dẫn đến trạng thái thắng.

8.2.2 8.2.2 Nim

Trò chơi có nhiều đồng; mỗi lượt lấy tùy ý từ một đồng, người không lấy được thì thua.

Lấy XOR kích thước các đồng; bằng 0 thì thua, khác 0 thì thắng.

Nước tối ưu là chọn đồng có bit cao nhất của XOR bật, rồi lấy sao cho XOR trở thành 0.

Ghi chú/công thức: $XOR \neq 0, 1 \quad XOR = 0$

Ghi chú/công thức: $0, 1$

8.2.3 8.2.3 Số Grundy

Trò chơi có số Grundy a tương đương một đồng Nim kích thước a .

Số Grundy của trạng thái là mex của các số Grundy của mọi trạng thái đi tới được.

Nếu tách thành nhiều trò chơi, dùng XOR các số Grundy.

Nếu có nhiều trò chơi và mỗi lượt chỉ đi một trò, lấy XOR.

Ghi chú/công thức: $1 - M \quad 2 \quad \text{mod } (M + 1)$

8.3 8.3 Ngày tháng

8.3.1 8.3.1 Đổi sang số ngày

Đổi sang số ngày, có xét năm nhuận.

```
1 int days(int y, int m, int d) {
2   if (m < 3) {
3     y--;
4     m += 12;
5   }
6   return 365 * y + y / 4 - y / 100 + y / 400 + (153 * m + 2) / 5 + d;
7 }
```

8.3.2 8.3.2 java.util.GregorianCalendar

Thứ trong tuần: [Sunday, Saturday] = [1,7].

Tháng: [January, December] = [0,11].

Sau ngày 4/10/1582 là ngày 15/10/1582; tại đây lịch Julius chuyển sang lịch Gregory.

Ghi chú/công thức: `setGregorianChange(new Date(Long.MIN_VALUE))`
 Ghi chú/công thức: `setGregorianChange(new Date(Long.MAX_VALUE))`
 Ghi chú/công thức:
 Ghi chú/công thức: $(\text{ } 2008 \quad 6 \quad 31 = 2008 \quad 7 \quad 1, 2008 \quad 7 \quad 31 - 1 = 2008 \quad 6 \quad 30)$
 Ghi chú/công thức: `getActualMaximum(Calendar.DAY_OF_MONTH)`
 Ghi chú/công thức: `isLeapYear(year)`

8.4 8.4 Mục nhỏ

8.4.1 8.4.1 Scanner

Chú ý đặc tả của `hasNext` khác nhau.

```
1 class Scanner {
2   BufferedReader br;
3   StringTokenizer st;
4   Scanner(InputStream in) {
5     br = new BufferedReader(new InputStreamReader(in));
6     eat("");
7   }
8   void eat(String s) {
9     st = new StringTokenizer(s);
10  }
11  String nextLine() {
12    try {
13      return br.readLine();
14    } catch (IOException e) {
15      throw new IOError(e);
16    }
17  }
18  boolean hasNext() {
19    while (!st.hasMoreTokens()) {
20      String s = nextLine();
21      if (s == null) return false;
22      eat(s);
23    }
24    return true;
25  }
26  String next() {
27    hasNext();
28    return st.nextToken();
29  }
30  int nextInt() {
31    return Integer.parseInt(next());
32  }
33  long nextLong() {
34    return Long.parseLong(next());
35  }
36  double nextDouble() {
37    return Double.parseDouble(next());
38  }
39 }
```

8.4.2 8.4.2 Liệt kê bit-combination nCk

Liệt kê theo thứ tự tăng dần.

```
1 for (int comb = (1 << k) - 1; comb < 1 << n; ) {
2   //...
3   int x = comb & -comb, y = comb + x;
4   comb = ((comb & ~y) / x >> 1) | y;
5 }
```

8.4.3 8.4.3 Sinh hoán vị

next permutation của C++.

Sinh hoán vị kế tiếp và trả về liệu nó có tồn tại hay không.

Ngay cả khi có nhiều giá trị bằng nhau, vẫn sinh không trùng.

Muốn sinh toàn bộ hoán vị thì hãy sắp xếp ban đầu.

```
1 boolean nextPermutation(int[] is) {
2     int n = is.length;
3     for (int i = n - 1; i > 0; i--) {
4         if (is[i - 1] < is[i]) {
5             int j = n;
6             while (is[i - 1] >= is[--j]);
7             swap(is, i - 1, j);
8             rev(is, i, n);
9             return true;
10        }
11    }
12    rev(is, 0, n);
13    return false;
14 }
```

8.4.4 8.4.4 Xúc xắc

Sinh toàn bộ trạng thái quay của khối lập phương.

```
1 class Dice {
2     //top, front, left, right, back, bottom
3     int[] is = new int[6];
4     Dice(int...is) {
5         this.is = is;
6     }
7     void rollX() {
8         roll(0, 1, 5, 4);
9     }
10    void rollY() {
11        roll(0, 3, 5, 2);
12    }
13    void rollZ() {
14        roll(1, 3, 4, 2);
15    }
16    void roll(int a, int b, int c, int d) {
17        int t = is[d];
18        is[d] = is[c];
19        is[c] = is[b];
20        is[b] = is[a];
21        is[a] = t;
22    }
23    Dice[] rollAll() {
24        Dice[] dices = new Dice[24];
25        for (int i = 0; i < 6; i++) {
26            for (int j = 0; j < 4; j++) {
27                dices[i * 4 + j] = new Dice(is.clone());
28                rollZ();
29            }
30            if ((i & 1) > 0) rollX();
31            else rollY();
32        }
33        return dices;
34    }
35 }
```


8.4.5 8.4.5 Số Josephus

Ghi chú/công thức: $n \quad m \quad k \quad [0, n - 1]$
 $J(n, m, 0) = -1$
 $J(n, m, k) = (J(n - 1, m, k - 1) + k) \% n$
 $O(k)$

```
1 int josephus(int n, int m, int k) {
2 int x = -1;
3 for (int i = n - k + 1; i <= n; i++) {
4 x = (x + m) % i;
5 }
6 return x;
7 }
```

Số Josephus ngược.

$O(n)$
Ghi chú/công thức: $n \quad m \quad x \quad [1, n]$

```
1 int invJosephus(int n, int m, int x) {
2 for (int i = n; ; i--) {
3 if (x == i) return n - i;
4 x = (x - m % i + i) % i;
5 }
6 }
```

8.4.6 8.4.6 Hình chữ nhật lớn nhất

Cho mảng chiều cao; tìm diện tích hình chữ nhật lớn nhất chứa trong đó.
 $O(n)$

```
1 int maxRect(int[] _hs) {
2 int n = _hs.length;
3 int res = 0;
4 int[] hs = new int[n + 1]; // n 番に番兵 0 を置く
5 System.arraycopy(_hs, 0, hs, 0, n);
6 Stack<Integer> sx = new Stack<Integer>();
7 Stack<Integer> sh = new Stack<Integer>();
8 sh.push(0);
9 for (int i = 0; i <= n; i++) {
10 int x = i;
11 while (sh.peek() > hs[i]) {
12 res = max(res, sh.pop() * (i - (x = sx.pop())));
13 }
14 sx.push(x);
15 sh.push(hs[i]);
16 }
17 return res;
18 }
```

9 9 Tìm kiếm

9.1 9.1 Tìm kiếm alpha-beta

```
1 int alphaBeta(State s, int alpha, int beta) {
2 if (s.finished()) return s.calcScore();
3 for (State t : s.next()) {
4 alpha = max(alpha, -alphaBeta(t, -beta, -alpha));
5 if (alpha >= beta) break;
6 }
7 return alpha;
8 }
```

9.2 9.2 Bài toán tô màu đỉnh tối thiểu

Tô màu bằng số màu nhỏ nhất sao cho hai đỉnh kề nhau không cùng màu.
Ý tưởng tìm kiếm.

1. Tách thành các thành phần liên thông rồi giải.
2. Sắp theo thứ tự MA rồi bắt đầu tìm kiếm.
3. DFS cố cắt tỉa theo số màu.

```
1 int[] coloring(boolean[] g) {
2     this.g = g;
3     int n = g.length;
4     res = new int[n];
5     id = new int[n + 1]; //元のインデックス (n 番は番兵)
6     int[] b = new int[n + 1]; //連結度
7     for (int i = 0; i <= n; i++) id[i] = i;
8     for (s = 0, t = 1; t <= n; t++) { //MA 順序でソート
9         int p = t; //最大連結度の点
10        for (int i = t; i < n; i++) {
11            if (g[id[t - 1]][id[i]]) b[id[i]]++;
12            if (b[id[p]] < b[id[i]]) p = i;
13        }
14        swap(id, t, p);
15        if (b[id[t]] == 0) { //連結度 0 なので [s,t) が連結成分
16            min = n + 1;
17            dfs(new int[n], s, 0);
18            s = t;
19        }
20    }
21    return res;
22 }
23 void dfs(int[] is, int p, int k) { //is:現在の塗り方,p:次塗る点,k:使った色の数
24     if (k >= min) return;
25     if (p == t) {
26         for (int i = s; i < t; i++) res[id[i]] = is[i];
27         min = k;
28     } else {
29         boolean[] used = new boolean[k + 1];
30         for (int i = 0; i < p; i++) if (g[id[p]][id[i]]) used[is[i]] = true;
31         for (int i = 0; i <= k; i++) if (!used[i]) { //今までに使った色 +1 まで調べる
32             int[] js = is.clone();
33             js[p] = i;
34             dfs(js, p + 1, max(k, i + 1));
35         }
36     }
37 }
```

9.3 9.3 Số matching hoàn hảo

Giải tổng quát các bài DP bit như đếm matching hoàn hảo hoặc đếm tập độc lập.

```
1 long match(V[] vs) {
2     int n = vs.length;
3     for (int i = 0; i < n; i++) vs[i].id = i;
4     for (V v : vs) for (V u : v) v.max = max(v.max, u.id);
5     long[] crt = {1};
6     int[] cvi = new int[n], civ = new int[n];
7     int cn = 0;
8     fill(cvi, -1);
9     for (int k = 0; k < n; k++) {
10        int nn = 0;
11        int[] nvi = new int[n], niv = new int[n];
12        fill(nvi, -1);
13        for (int i = 0; i <= k; i++) if (vs[i].max > k) {
14            niv[nn] = i;
15            nvi[i] = nn++;
16        }
17    }
```

```

16 }
17 long[] next = new long[1 << nn];
18 loop : for (int i = 0; i < 1 << cn; i++) if (crt[i] > 0) {
19 int i2 = 0;
20 boolean need = false;
21 for (int j = 0; j < cn; j++) if ((i >> j & 1) != 0) {
22 if (nvi[civ[j]] < 0) {
23 if (need) continue loop;
24 need = true;
25 } else {
26 i2 |= 1 << nvi[civ[j]];
27 }
28 }
29 if (need) {
30 next[i2] += crt[i];
31 } else {
32 if (nvi[k] >= 0) {
33 next[i2 | 1 << nvi[k]] += crt[i];
34 }
35 for (V u : vs[k]) if (u.id < k && (i >> cvi[u.id] & 1) != 0) {
36 next[i2 ^ 1 << nvi[u.id]] += crt[i];
37 }
38 }
39 }
40 crt = next; cvi = nvi; civ = niv; cn = nn;
41 }
42 return crt[0];
43 }
44 class V extends ArrayList<V> {
45 int id, max = -1;
46 }

```

10 10 Trình trực quan hóa

```

1 class Vis extends JFrame {
2 BufferedImage img;
3 Graphics2D g;
4 boolean stop;
5 Vis() {
6 img = new BufferedImage(500, 500, BufferedImage.TYPE_INT_RGB);
7 g = (Graphics2D)img.getGraphics();
8 g.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
9 RenderingHints.VALUE_ANTIALIAS_ON);
10 clear();
11 JComponent c = new JComponent() {
12 protected void paintComponent(Graphics g) {
13 super.paintComponent(g);
14 g.drawImage(img, 0, 0, null);
15 }
16 };
17 c.setPreferredSize(new Dimension(500, 500));
18 add(c);
19 addMouseListener(new MouseAdapter() {
20 public void mouseClicked(MouseEvent e) {
21 stop = false;
22 }
23 });
24 setDefaultCloseOperation(EXIT_ON_CLOSE);
25 pack();
26 setVisible(true);
27 }
28 void vis() {
29 repaint();
30 stop = true;
31 try {
32 while (stop) Thread.sleep(10);

```

```

33 } catch (Exception e) {
34 }
35 }
36 void clear() {
37 g.setColor(Color.WHITE);
38 g.fillRect(0, 0, 500, 500);
39 g.setColor(Color.BLACK);
40 }
41 }

```

11 11 Bảng tiện dụng

11.1 11.1 Định dạng

'-': căn trái.
 '+': luôn in dấu.
 ' ': thêm khoảng trắng trước số không âm.
 '0': điền 0 ở bên trái.
 ',': phân tách mỗi 1000.

11.2 11.2 Biểu thức chính quy

11.2.1 11.2.1 Lớp ký tự

[abc]: a, b hoặc c (lớp đơn giản).
 [^abc]: ký tự không phải a, b, c (phủ định).
 Ghi chú/công thức: [a-zA-Z] a - z A - Z ()
 Ghi chú/công thức: [a-d[m-p]] a - d m - p:[a-dm-p] ()
 [a-z&&[def]]: d, e, f (giao).
 Ghi chú/công thức: [a-z&&[^bc]] b c a - z:[ad-z] ()
 Ghi chú/công thức: [a-z&&[^m-p]] m - p a - z:[a-lq-z] ()
 .: ký tự bất kỳ.
 \d: chữ số [0-9].
 \D: không phải chữ số [^0-9].
 \s: ký tự trắng [\t\n\x0B\f\r].
 \S: không phải ký tự trắng [^\s].
 \w: ký tự từ [a-zA-Z_0-9].
 \W: không phải ký tự từ [^\w].

11.2.2 11.2.2 Biên

^: đầu dòng.
 \$: cuối dòng.
 \b: biên từ.
 \B: không phải biên từ.
 \A: đầu vào.
 \G: cuối lần match trước.
 \Z: cuối đầu vào, trừ ký hiệu xuống dòng cuối nếu có.
 \z: cuối đầu vào.

11.2.3 11.2.3 Lượng từ

Không chỉ định gì thì match dài nhất.
 Thêm ? phía sau thì match ngắn nhất.

Thêm + phía sau thì trở thành possessive/greedy cứng, không quay lui khi match thất bại.

$X?$: X xuất hiện 1 hoặc 0 lần.

X^* : X xuất hiện 0 lần trở lên.

X^+ : X xuất hiện 1 lần trở lên.

$X\{n\}$: X xuất hiện n lần.

$X\{n,\}$: X xuất hiện ít nhất n lần.

$X\{n,m\}$: X xuất hiện từ n đến m lần.

11.2.4 11.2.4 Khác

Nhóm capture số 0 là toàn bộ match; các nhóm được đánh số từ 1 theo thứ tự mở ngoặc.

XY : Y ngay sau X.

$X|Y$: X hoặc Y.

(X) : X, nhóm capture.

$(?:X)$: X, nhóm không capture.

$\backslash n$: nhóm capture thứ n đã match.

$(?i)$: bật CASE_INSENSITIVE.

$(?-i)$: tắt CASE_INSENSITIVE.

11.3 11.3 Bảng số

n $\log_{10} n$ $n!$ $nC(n/2)$ $LCM(1 \dots n)$ P_n B_n

2	0.30	2	2	2	2	2
3	0.48	6	3	6	3	5
4	0.60	24	6	12	5	15
5	0.70	120	10	60	7	52
6	0.78	720	20	60	11	203
7	0.85	5040	35	420	15	877
8	0.90	40320	70	840	22	4140
9	0.95	362880	126	2520	30	21147
10	1.00	3628800	252	2520	42	115975
11	39916800	462	27720	56	678570	
12	479001600	924	27720	77	4213597	
15	6435	360360	176	1382958545		
20	184756	232792560	627			
25	5200300	1958				
30	155117520	5604				
40	37338					
50	204226					
70	4087968					
100	190569292					